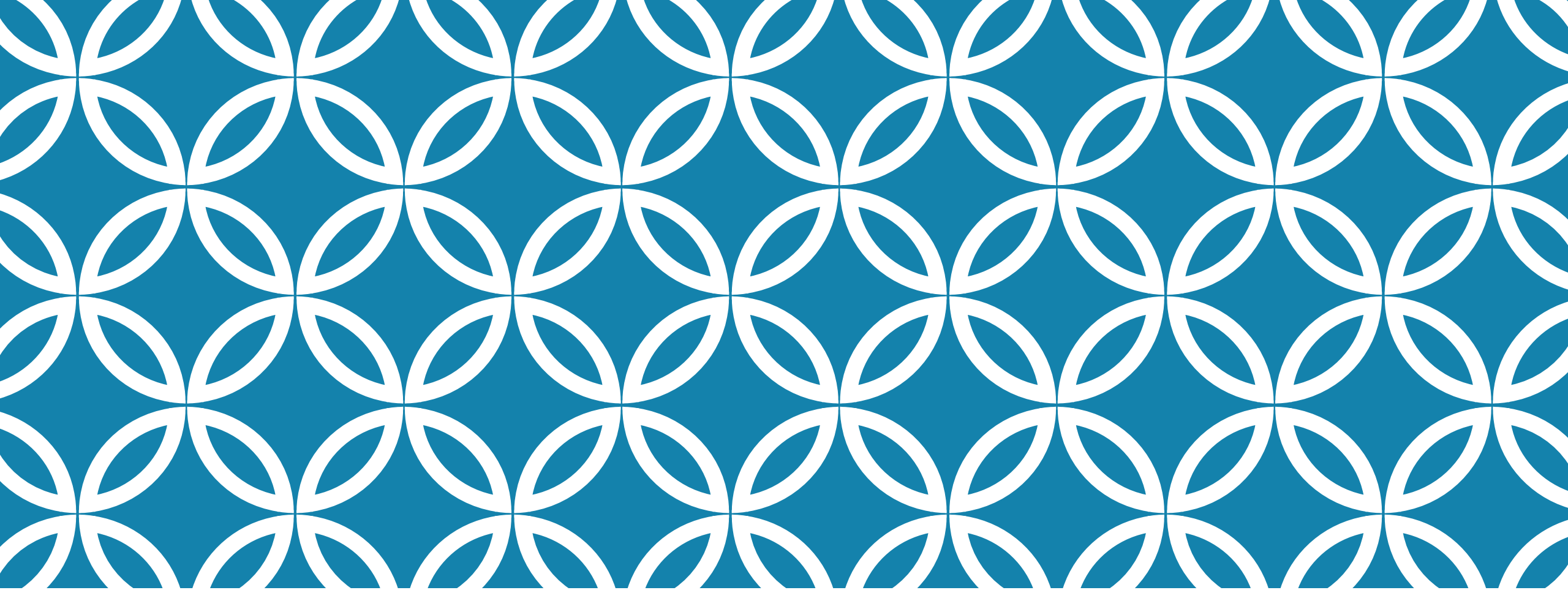


NODE.JS

Part II

NEED FOR A TEMPLATE

- If you want to add specific handling for different HTTP verbs (e.g. GET, POST, DELETE, etc.)
- Separately handle requests at different URL paths ("routes")
- Serve static files, or use templates to dynamically create the response,
- Node won't be of much use on its own.
- You will either need to write the code yourself, or you can avoid reinventing the wheel and use a web framework!

APPLICATION FRAMEWORKS

- provides frozen spots
  - overall architecture
  - How the components interact
- allows to concentrate in hot spots to extend the behaviour of the framework
  - Hot spots are the functions written for the application
- A framework is not suitable for a problem when …

WEB APPLICATION FRAMEWORKS

- An application framework that is designed to support development of web applications that generally includes
  - Database support
  - Templating framework for generating dynamic web content
  - HTTP session management with middleware support
  - Built-in testing framework
- It can also support internationalization, security and privacy
- Consistent look and feel and consistent with database

WEB FRAMEWORKS EXAMPLES

- Ruby on Rails
- Play
- ASP.NET
- Django
- Symfony
- Spring
- Vue.js
- Angular js

EXPRESS

- Express is the most popular *Node* web framework, and is the underlying library for a number of other popular Node web frameworks.
- It provides mechanisms to:
- Write handlers for requests with different HTTP verbs at different URL paths (routes).
- Integrate with "view" rendering engines in order to generate responses by inserting data into templates.
- Set common web application settings like the port to use for connecting, and the location of templates that are used for rendering the response.
- Add additional request processing "middleware" at any point within the request handling pipeline.

EXPRESS FEATURES

- ❑ *Express* itself is fairly minimalist
- ❑ Developers have created compatible middleware packages to address almost any web development problem
- ❑ There are libraries to work with cookies, sessions, user logins, URL parameters, POST data, security headers, and *many* more.
- ❑ You can find a list of middleware packages maintained by the Express team at Express Middleware (along with a list of some popular 3rd party packages).

IS EXPRESS OPINIONATED

- Opinionated frameworks are those with opinions about the "right way" to handle any particular task. They often support rapid development *in a particular domain* (solving problems of a particular type) because the right way to do anything is usually well-understood and well-documented

- Unopinionated frameworks, by contrast, have far fewer restrictions on the best way to glue components together to achieve a goal, or even what components should be used.

EXPRESS OPINION

- Express is unopinionated.
- You can insert almost any compatible middleware you like into the request handling chain, in almost any order you like.
- You can structure the app in one file or multiple files, using any directory structure.
- You may sometimes feel that you have too many choices

EXPRESS FEATURES

- Express provides methods to specify what function is called for a particular HTTP verb (GET, POST, SET, etc.) and URL pattern ("Route")
- Provides methods to specify what template ("view") engine is used
- Where template files are located
- What template to use to render a response
- You can use Express middleware to add support for cookies, sessions, and users, getting POST/GET parameters, etc.
- You can use any database mechanism supported by Node (Express does not define any database-related behavior).

EXPRESS APP

- This object, which is traditionally named app, has methods for
- routing HTTP requests
- configuring middleware,
- rendering HTML views,
- registering a template engine, and
- modifying application settings that control how the application behaves (e.g. the environment mode, whether route definitions are case sensitive, etc.)
- res.json() to send a JSON response or res.sendFile() to send a file

```
const express = require("express");
const app = express();
const port = 3000;

app.get("/", function (req, res) {
  res.send("Hello World!");
});

app.listen(port, function () {
  console.log(`Example app listening on
port ${port}!`);
});
```

ROUTER

A common convention for Node and Express is to use error-first callbacks. In this convention, the first value in your *callback functions* is an error value, while subsequent arguments contain success data.

Routes allow you to match particular patterns of characters in a URL, and extract some values from the URL and pass them as parameters to the route handler

The node:path module provides utilities for working with file and directory paths

The path.join() method joins all given path segments together using the platform-specific separator as a delimiter, then normalizes the resulting path.

Zero-length path segments are ignored. If the joined path string is a zero-length string then '.' will be returned, representing the current working directory.

```
path.join('/foo', 'bar', 'baz/asdf', 'quux', '..');

// Returns: '/foo/bar/baz/asdf'
```

REQUEST HANDLING

- The *Express application* object also provides methods to define route handlers for all the other HTTP verbs, which are mostly used in exactly the same way:
- checkout(), copy(), **delete()**, **get()**, head(), lock(), merge(), mkactivity(), mkcol(), notify(), options( )patch(), **post()**, purge(), **put()**, report(), search(), subscribe(), trace(), unlock(), unsubscribe()
- There is a special routing method, app.all(), which will be called in response to any HTTP method.
- This is used for loading middleware functions at a particular path for all request methods.

```
app.all("/secret", function (req, res, next) {
        console.log("Accessing the secret section…");
         next(); // pass control to the next handler });
```

ROUTING THROUGH EXPRESS

```
const wiki = require("./wiki.js");

app.use("/wiki", wiki);
```

```
const router = express.Router();
// Home page route
router.get("/", function (req, res) {
  res.send("Wiki home page");
});
```

- Often it is useful to group route handlers for a particular part of a site together and access them using a common route-prefix
- a site with a Wiki might have all wiki-related routes in one file and have them accessed with a route prefix of */wiki/*
- In *Express* this is achieved by using the express.Router object.

https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/Introduction

ALL ABOUT ERRORS

- Errors are handled by one or more special middleware functions that have four arguments, instead of the usual three: (err, req, res, next)

```
app.use(function (err, req, res, next) {
    console.error(err.stack);
    res.status(500).send("Something broke!"); });
```

These can return any content required, but must be called after all other app.use() and routes calls so that they are the last middleware in the request handling process!

TEMPLATES

- Template engines (also referred to as "view engines") allow you to specify the *structur*e of an output document in a template, using placeholders for data that will be filled in when a page is generated.

- Templates are often used to create HTML, but can also create other types of documents

- At runtime, the template engine replaces variables in a template file with actual values, and transforms the template into an HTML file sent to the client

HTML TEMPLATE ENGINES

- Some popular template engines that work with Express are Pug, Mustache, and EJS.
- The Express application generator uses Pug as its default, but it also supports Handlebars, and EJS, among others.
- In your application settings code you set the template engine to use and the location where Express should look for templates using the 'views' and 'view engine' settings
- Directory to keep the template files

```
app.set('views', './views')
```

- To set pug as the template engine

```
app.set('view engine', 'pug')
```

- The view engine cache does not cache the contents of the template's output, only the underlying template itself. The view is still re-rendered with every request even when the cache is on.

PUG ENGINE

```
app.get("/views", function
(req, res) {
  res.send("Hello World!");
});
```

```
html
  head
    title= title
  body
    h1= message
    a(href = url) Homepage
```

```
app.get('/views', (req, res) => {
  res.render('index.pug', { title: 'Hey',
message: 'Hello there!' })
})
```

SETTING CONDITIONALS

```
router.get("/about", function (req, res) {
 res.render('signup.pug', {message: 'Wiki
home page',
url:"http://www.tutorialspoint.com"});
});
```

```
router.get("/about", function (req, res) {
 res.render('signup.pug', {{user: {name:
"Aritra", age: "22"}
});
```

```
html
    head
        title Simple template
    body
        if(user)
            h1 Hi, #{user.name}
        else
            a(href = "/wiki") Sign Up
```

ROUTING THROUGH THE FOLDERS

- In pug file insert the image source

```
img(src = '/testImage.jpg' alt = 'Testing Image’)
```

- Sending values in URL
- http://localhost:3000/users/34/books/8989

```
app.get("/users/:userId/books/:bookId", (req, res) => {
                                    res.send(req.params.userId);
                                    });
```

- Insert static content

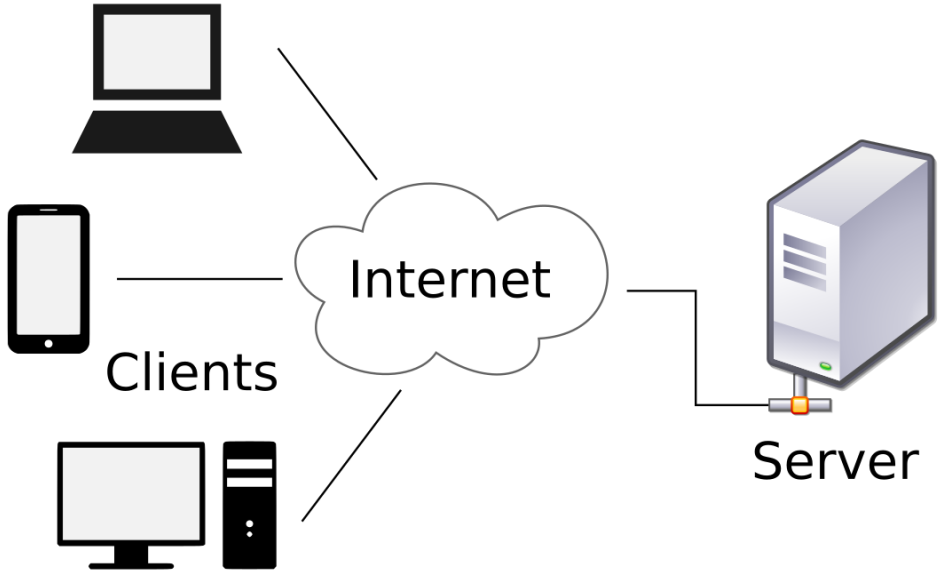
```
app.use(express.static('views’));
```

Any files in the public directory are served by adding their filename

- Adding a virtual path for the static content

```
app.use('/static', express.static('views'));
```

HTTP IS CLIENT DRIVEN

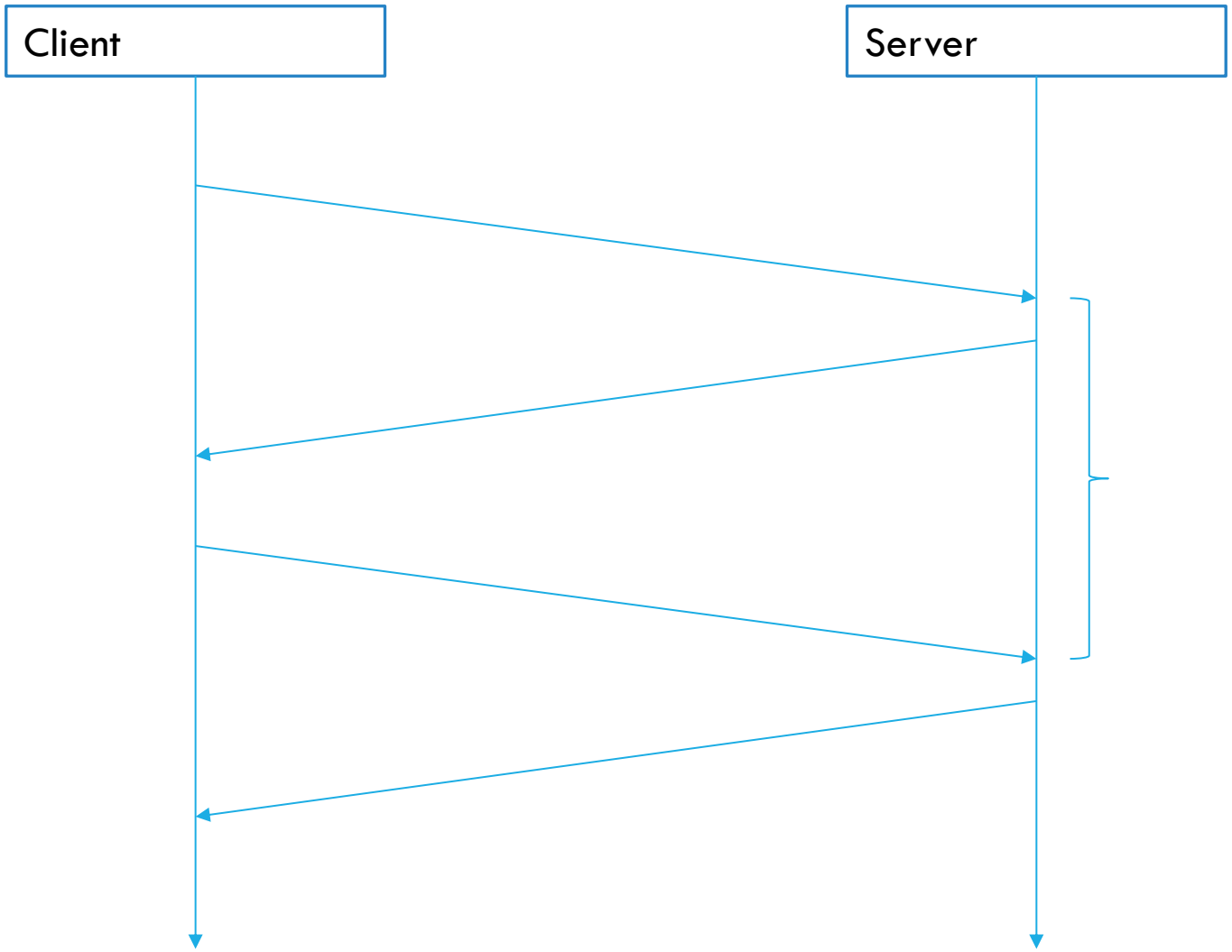


If client 2 updates important data that client 1 just pulled in, unless client 1 makes a second request for an update the server cannot notify client 1

Server is not able to push data as and when needed

PUSHING DATA

- Manual- User may click on “refresh” when (s)he has decided to pull the data
- Every time the window is opened data is pulled from the server
- Periodic update
  - Overhead on the server processes as it needs to process the request even when no update is to be notified
  - Polling
  - Exponential backoff



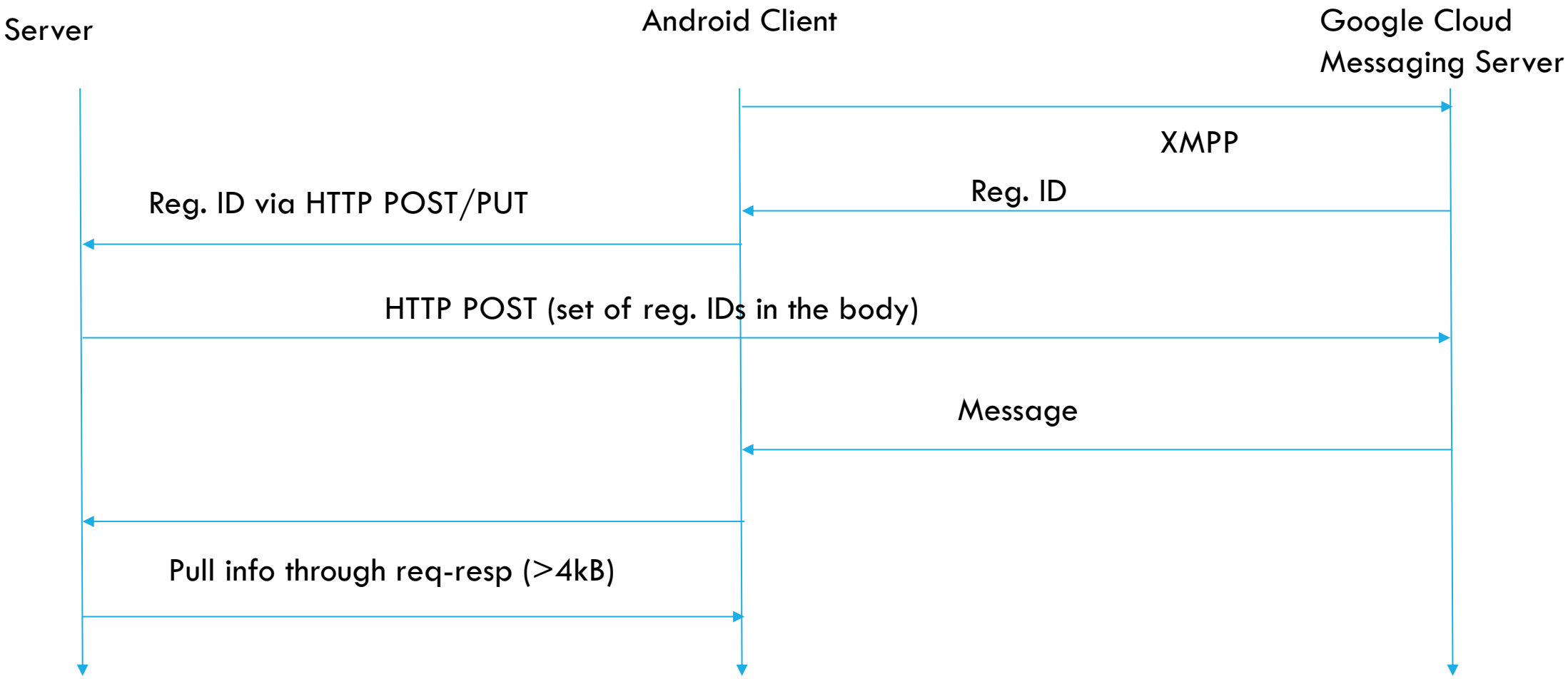
PERIODIC UPDATE

https://dzone.com/articles/thoughts-on-server-sent-events-http2-and-envoy-1

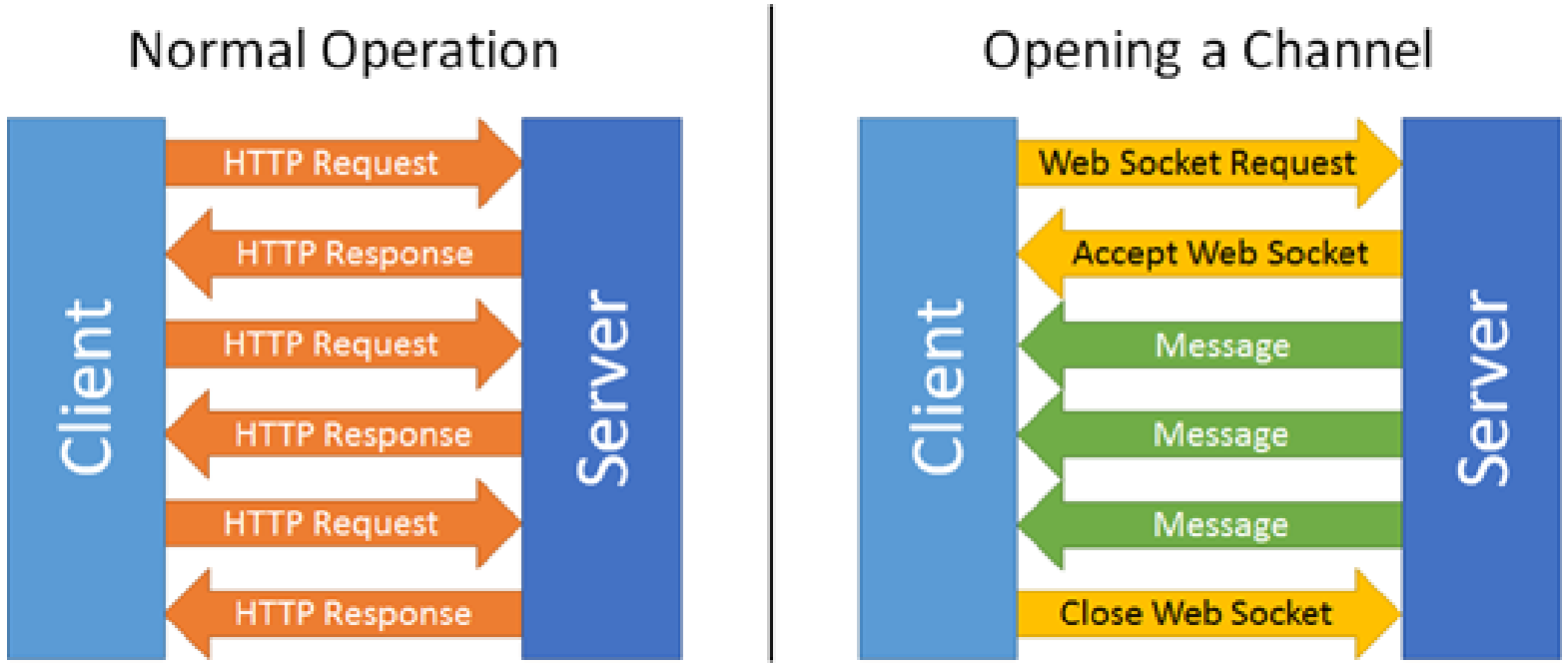
INTERMITTENT INTERNET CONNECTION

- How to handle intermittent internet connections
  - Event mechanisms and notifications should be designed to detect disconnection and reconnection
  - Periodically reconnect/ or getting an event from Android

PUSH NOTIFICATION

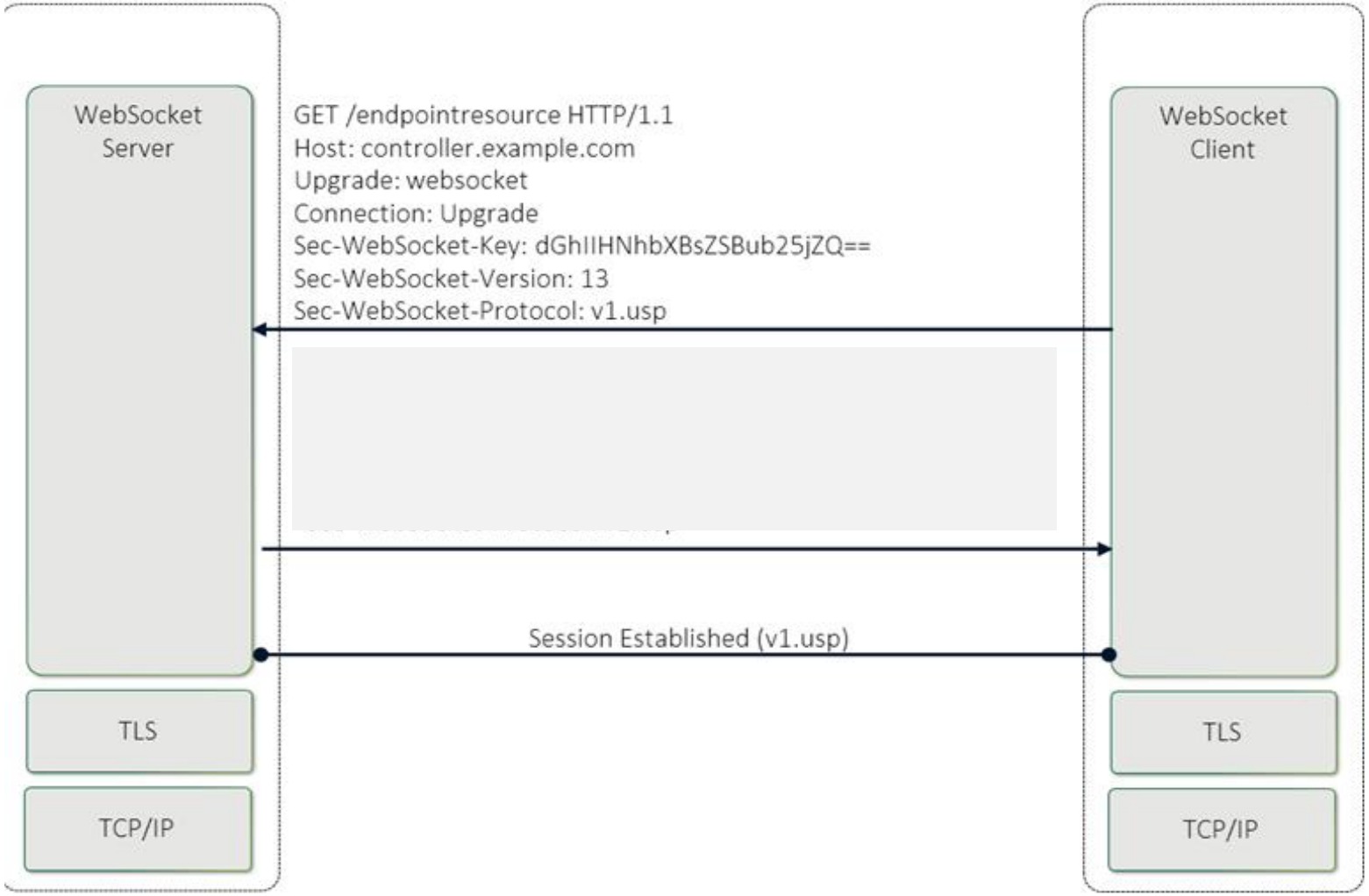


WEBSOCKETS



- communication protocol which features bi-directional, full-duplex communication over a persistent TCP connection
- Any party can push data anytime
- Single TCP connection for full duplex traffic
- Message transfer on websockets does not require all parts of HTTP to be sent (header, URL, content type, body etc.)
- Simply send binary messages or some other format back and forth in a server
- By default, port 80 is used
- Port 443 is used for connection tunneled over the TLS

WEB SOCKET HTTP COMPATIBILITY



WEB SOCKET ADVANTAGES

- Stateful connection
- Message overhead of polling is more than web socket
- STOMP- Simple Text Oriented Messaging Protocol

https://spring.io/guides/gs/messaging-stomp-websocket/

S. El Mimouni and M. Bouhdadi, "Formal modeling of the Simple Text Oriented Messaging Protocol using Event-B method," 2015 IEEE/ACS 12th International Conference of Computer Systems and Applications (AICCSA), 2015, pp. 1-4, doi: 10.1109/AICCSA.2015.7507170.

WEB SOCKET PROBLEMS

- Web sockets enable a server to push data only if the client is connected
- Web sockets are difficult to synchronize with more clients
- Keeping an open connection can have substantial resource impact
- For shared hosting servers, web socket is not a scalable option
- http responses can be cached by browser or by proxies
  - There is no such built-in mechanism for requests sent via webSockets

WEBSOCKET IN NODE.JS

- To communicate using the WebSocket protocol, we need to create a WebSocket object.
- The WebSocket constructor takes one mandatory argument — the URL of the WebSocket server to connect to.
- The constructor will throw a SecurityError if the destination doesn't allow access.
- The constructor takes another optional argument protocols, which allows a single server to implement multiple sub-protocols.
- Once the connection is established, the open event is fired, and after this point the socket is able to transmit data.
- On an error, the connection is closed and the close event will be fired.

https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API/Writing_WebSocket_client_applications

CONNECTION ESTABLISHMENT

```
const WebSocketServer =
require('ws').Server

const wss = new
WebSocketServer({port: 3000})
```

```
var ws = new
WebSocket('ws://localhost:3000');
```

SENDING AND RECEIVING MESSAGES

- The send() method is asynchronous: it does not wait for the data to be transmitted before returning to the caller.
- It just adds the data to its internal buffer and begins the process of transmission.
- The WebSocket.bufferedAmount property represents the number of bytes that have not yet been transmitted.
- To receive messages from the server, we listen for the message event.
- Our message event handler logs the received message, and increments our count of the number of message exchanges that have occurred

```
websocket.addEventListener("message", (e) => {

                    console.log(`RECEIVED: ${e.data}: ${counter}`);

                    counter++; });
```

EVENT HANDLING

```
websocket.addEventListener("<name of
event>", (<any parameters received>) =>
{ <event handler });
```

- the server can also send JSON strings, which the client can then parse into an object

```
websocket.addEventListener("message", (e) => { const
                                message = JSON.parse(e.data);
                                Console.log(`RECEIVED: ${message.iteration}:
                                ${message.content}`); counter++; });
```

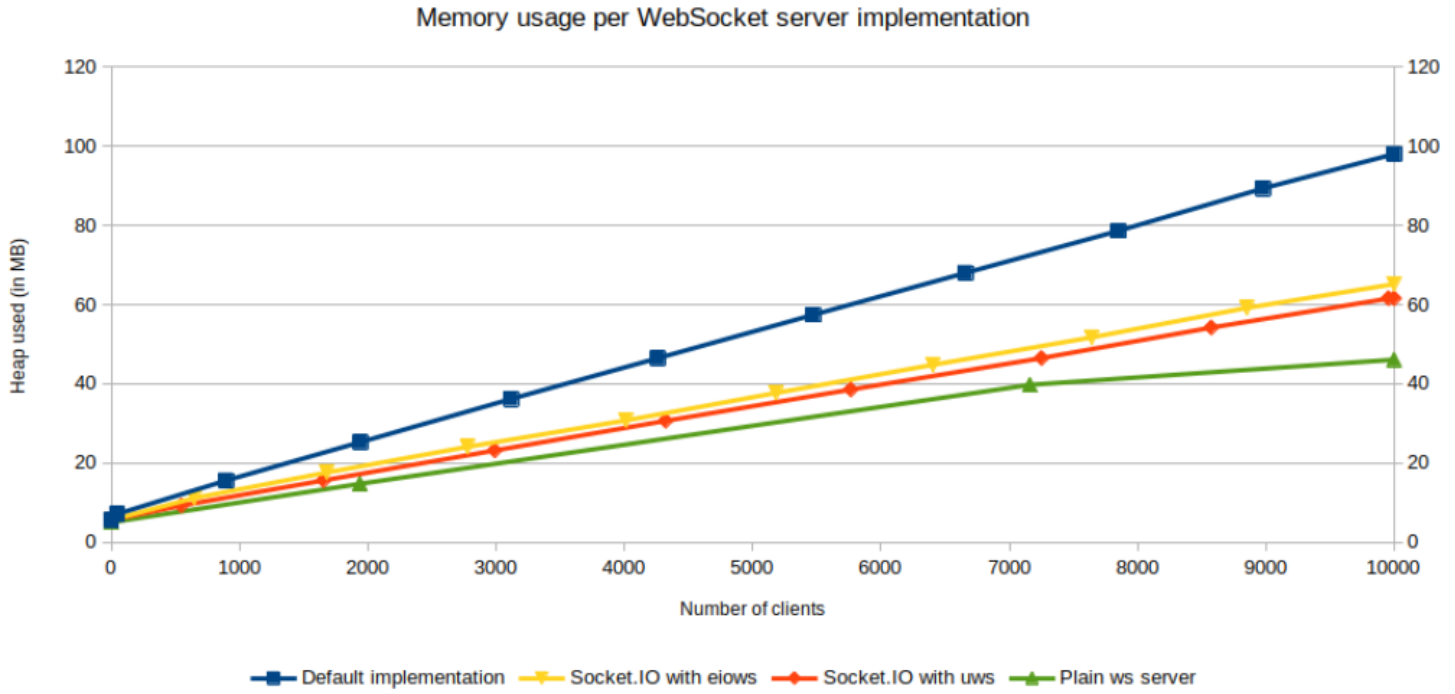
- When the connection is closed, because either the client or the server closed it or because an error occurred, the close event will be fired

```
websocket.addEventListener("close", () => {
                                Console.log("DISCONNECTED");
                                clearInterval(pingInterval); });
```

SOCKET.IO

- Socket.IO is a library that enables **low-latency**, **bidirectional** and **event-based** communication between a client and a server.
- The bidirectional channel between the Socket.IO server (Node.js) and the Socket.IO client (browser) is established with a WebSocket connection whenever possible, and will use HTTP long-polling as fallback.
- Although Socket.IO indeed uses WebSocket for transport when possible, it adds additional metadata to each packet. That is why a WebSocket client will not be able to successfully connect to a Socket.IO server

SOCKET.IO



- Under some particular conditions, the WebSocket connection between the server and the client can be interrupted with both sides being unaware of the broken state of the link.
- That's why Socket.IO includes a heartbeat mechanism, which periodically checks the status of the connection.
- And when the client eventually gets disconnected, it automatically reconnects with an exponential back-off delay, in order not to overwhelm the server.

WRITING A CHAT APPLICATION

- Create a package.json file to store all the dependency

```
{
"name": "socket-chat-example",
"version": "0.0.1",
"description": "my first socket.io app",
"dependencies": {}
}
```

- dependencies will be populated automatically in the json file and packages will be installed in a sub directory called node_modules
- `npm install express`
- sample index.js to create a server
- `npm install -g nodemon`

SOCKET.IO

- Express initializes app to be a function handler that you can supply to an HTTP server
- We define a route handler / that gets called when we hit our website home.
- We make the http server listen on port 3000.

```
const express = require('express');
const app = express();
const http = require('http');
const server = http.createServer(app);

app.get('/', (req, res) => {
  res.send('<h1>Hello world</h1>');
});

server.listen(3000, () => {
  console.log('listening on *:3000');
});
```

https://socket.io/get-started/chat#:~:text=The%20main%20idea%20behind%20Socket,binary%20data%20is%20supported%20too.

SOCKET.IO

```
app.get('/', (req, res) => {
    res.sendFile(__dirname +
'/index.html');
});
```

- Socket.IO is composed of two parts:
- A server that integrates with (or mounts on) the Node.JS HTTP Server socket.io
- A client library that loads on the browser side socket.io-client

```
<html>
<head>Add CSS</head>
<body>
    <ul id="messages"></ul>
    <form id="form" action="">
      <input id="input" autocomplete="off"
      /><button>Send</button>
    </form>
</body>
</html>
```

SOCKET.IO

```
<script src="/socket.io/socket.io.js"></script>
<script>
var socket = io();
</script>
```

A client library that loads on the browser side socket.io-client

A new instance of socket.io by passing the server (the HTTP server) object

```
const express = require('express');
const app = express();
const http = require('http');
const server = http.createServer(app);
const { Server } = require("socket.io");
const io = new Server(server);
```

A server that integrates with (or mounts on) the Node.JS HTTP Server socket.io

- To indicate connected users the index.js is updated as follows.

```
io.on('connection', (socket) => {
    console.log('a user connected');
});
```

CLIENT AND SERVER

```
<script src="/socket.io/socket.io.js"></script>
<script>
var socket = io();
</script>
```

- index.html should be modified to include the io client
- If you would like to use the local version of the client-side JS file, you can find it at node_modules/socket.io/client-dist/socket.io.js.
- No URLs are specified when io() is called, since it defaults to trying to connect to the host that serves the page
- loading socket.io client from content delivery network

```
<script src="https://cdn.socket.io/4.5.0/socket.io.min.js"
integrity="…" crossorigin="anonymous"></script>
```

- events can be added for individual sockets

```
io.on('connection', (socket) => {
console.log('a user connected');
socket.on('disconnect', () => {
console.log('user disconnected');
});
});
```

CLIENT SENDS DATA

- The main idea behind Socket.IO is that you can send and receive any events you want, with any data you want. Any objects that can be encoded as JSON will do, and binary data is supported too.
- Let's make it so that when the user types in a message, the server gets it as a chat message event

```
<html>
<head>Add CSS</head>
<body>
      <ul id="messages"></ul>
      <form id="form" action="">
        <input id="input" autocomplete="off"
        /><button>Send</button>
      </form>
</body>
</html>
```

```
<script src="/socket.io/socket.io.js"></script>
<script>
  var socket = io();

  var form = document.getElementById('form');
  var input = document.getElementById('input');

  form.addEventListener('submit', function(e) {
    e.preventDefault();
    if (input.value) {
      socket.emit('chat message', input.value);
      input.value = '';
    }
  });
</script>
```

```
io.on('connection', (socket) => {
  socket.on('chat message', (msg) => {
    console.log('message: ' + msg);
  });
});
```

});

BROADCAST A MESSAGE

```
io.emit('some event', {Property1: 'value1', Property2: 'value2' });
```

- Broadcasts a data to everyone including the sender
- If you want to send a message to everyone except for a certain emitting socket, we have the broadcast flag for emitting from that socket:

```
io.on('connection', (socket) => {
socket.broadcast.emit('hi');
```

To broadcast the received message, at index.js the following is included

```
io.on('connection', (socket) => {
  socket.on('chat message', (msg) => {
    io.emit('chat message', msg);
  });
});
```

RECEIVING MESSAGE FROM OTHERS

on the client side when we capture a chat message event we'll include it in the page

```
<script>
  var socket = io();

  var messages = document.getElementById('messages');
  var form = document.getElementById('form');
  var input = document.getElementById('input');

  form.addEventListener('submit', function(e) {
    e.preventDefault();
    if (input.value) {
      socket.emit('chat message', input.value);
      input.value = '';
    }
  });

  socket.on('chat message', function(msg) {
    var item = document.createElement('li');
    item.textContent = msg;
    messages.appendChild(item);
    window.scrollTo(0, document.body.scrollHeight);
  });
</script>
```

https://socket.io/get-started/chat

ACKNOWLEDGEMENT

Client

```
socket.timeout(5000).emit('event',
{ foo: 'bar' }, 'baz', (err,
response) => {
  if (err) {
    // the server did not
acknowledge the event in the given
delay
  } else {
    console.log(response.status);
// 'ok'
  }
});
```

Server

```
io.on('connection', (socket) => {
  socket.on('event', (arg1, arg2,
callback) => {
    console.log(arg1); // { foo:
'bar' }
    console.log(arg2); // 'baz'
    callback({
      status: 'ok'
    });
  });
});
```

LISTENING TO ALL

```
socket.on('set users',
function(msg){

   var item =
document.createElement('li’);

  item.textContent = msg;

messages.appendChild(item);

window.scrollTo(0,
document.body.scrollHeight);

        });
```

```
socket.onAny((eventName, ...args) =>
{

   var item =
document.createElement('li');

item.textContent = args;

messages.appendChild(item);

window.scrollTo(0,
document.body.scrollHeight);

        });
```

ROOMS

A *room* is an arbitrary channel that sockets can join and leave. It can be used to broadcast events to a subset of connected clients

```
io.on('connection', (socket) => {
// join the room named 'some room'
socket.join('BCSEIII group');

// broadcast to all connected clients in the room
io.to('BCSEIII group').emit('Assignments', 'Assignment 3');

// broadcast to all connected clients except those in the room
io.except(BCSEIII group ').emit('hello', 'world');

// leave the room
socket.leave(' BCSEIII group ');
});
```